POC-05 Enterprise RAG

Microsoft Foundry + Azure AI Search

Beginner Reference Guide

Manual Azure setup, Python runbook, verification, screenshots, issues and fixes

Completed POC path

Synthetic documents -> chunking -> embeddings -> Azure AI Search vector/hybrid retrieval -> Foundry chat model -> grounded answer with citations -> FastAPI.

How to use this guide

Follow the sections in order when rebuilding the POC. Use the troubleshooting pages when the portal or Python output differs.

Part A - Azure setup

Prerequisites, resource group, Foundry resource/project, chat model, embedding model, Azure AI Search.

Part B - Local configuration

Project structure, environment setup, dependencies, .env values and configuration validation.

Part C - Run and verify

Unit tests, ingestion, vector/hybrid retrieval, grounded RAG, evaluation, failure tests and smoke test.

Part D - API and operations

FastAPI, Swagger verification, observability, keyless security, GitHub safety and cleanup.

Part E - Troubleshooting

Every important issue encountered in this POC is recorded with cause, fix and verification.

Contents - Azure and local setup

- POC objective and architecture
- Recommended resource names and cost guardrails
- Windows prerequisites and Azure CLI login
- Create and verify resource group
- Create Microsoft Foundry resource and project
- Deploy chat model and handle model quota/region problems
- Deploy text-embedding-3-small and capture endpoint/key
- Create Azure AI Search and capture endpoint/admin key
- Build the final .env file
- Install dependencies and run verify_config.py

Contents - RAG execution and verification

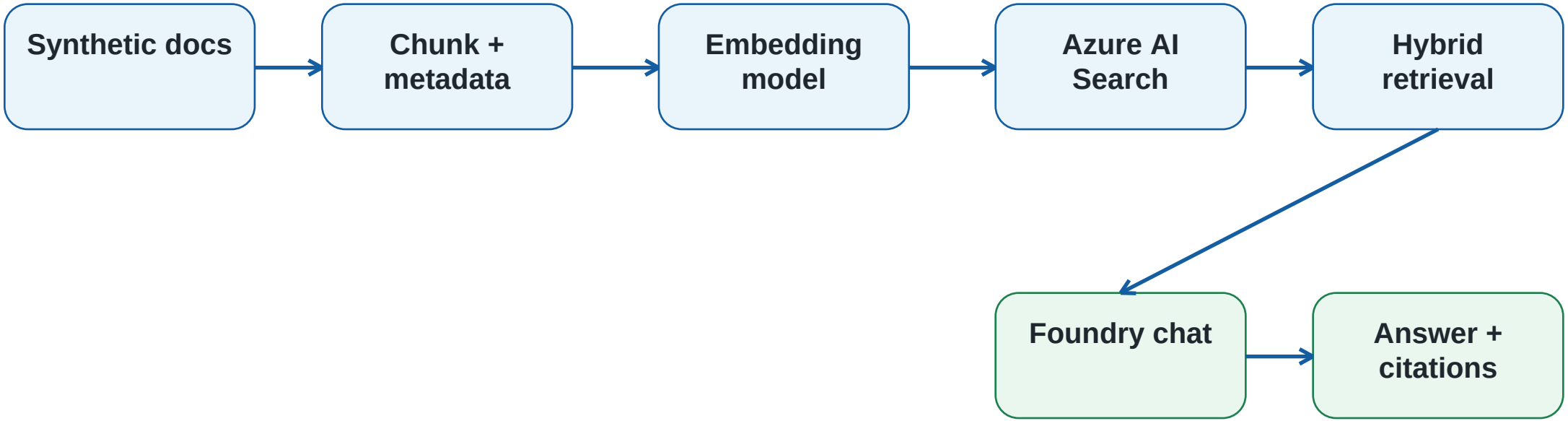
- Run local unit tests
- Run rag.ingest and verify Azure Search index
- Run vector retrieval and hybrid retrieval
- Test metadata filtering
- Run grounded RAG question and unsupported question
- Run 10-question evaluation and failure tests
- Run smoke_test.py
- Start FastAPI and test /health, /retrieve and /ask in Swagger
- Application Insights and keyless/managed identity options
- GitHub safety, cleanup and final completion checklist
- Interview revision questions

POC objective

Build a small Retrieval-Augmented Generation application over safe synthetic business documents using Microsoft Foundry and Azure AI Search.

- Create a curated corpus of 10-30 short synthetic documents; this project includes 12.
- Chunk text and attach metadata such as source, category, chunk_id and effective date.
- Generate embeddings with a Foundry embedding deployment.
- Store text plus vectors in Azure AI Search.
- Compare vector-only retrieval with hybrid keyword + vector retrieval.
- Generate an answer from retrieved context only and return source identifiers.
- Evaluate retrieval quality and test unsupported, ambiguous, conflicting and prompt-injection cases.
- Expose the workflow through a local FastAPI application.

Architecture - end-to-end data flow



Why hybrid retrieval?

Keyword search is strong for exact terms while vector search captures semantic similarity. The POC runs both and compares the results.

Recommended resource names

Use an isolated resource group so the whole POC can be deleted safely after testing.

Resource group	rg-poc05-foundry-rag
Foundry project	poc05-rag-project or the final regional project
Foundry resource	portal-generated / project resource
Azure AI Search	search-poc05-rag-<unique>
Search index	poc05-rag-index
Chat deployment	Recommended alias: rag-chat; completed run used Phi-4-mini-instruct
Embedding deployment	Recommended alias: rag-embedding; completed run used text-embedding-3-small
Application Insights	appi-poc05-rag (optional)

Windows prerequisites and Azure login

- Python 3.11 or 3.12
- Git
- Azure CLI
- VS Code (recommended)

```
python --version
git --version
az version
az login
az account show
```

If az is not recognized

Azure CLI is a separate Windows program. Installing azure-identity with pip does not install the az command. Install Azure CLI, reopen the terminal, and restart Windows if PATH still has not refreshed.

Step 1 - Create the resource group

1. Open Azure Portal and search for Resource groups.
2. Select Create.
3. Choose the Azure subscription used for this POC.
4. Enter resource group name rg-poc05-foundry-rag.
5. Choose a region suitable for the Foundry resources you plan to deploy.
6. Select Review + create, then Create.

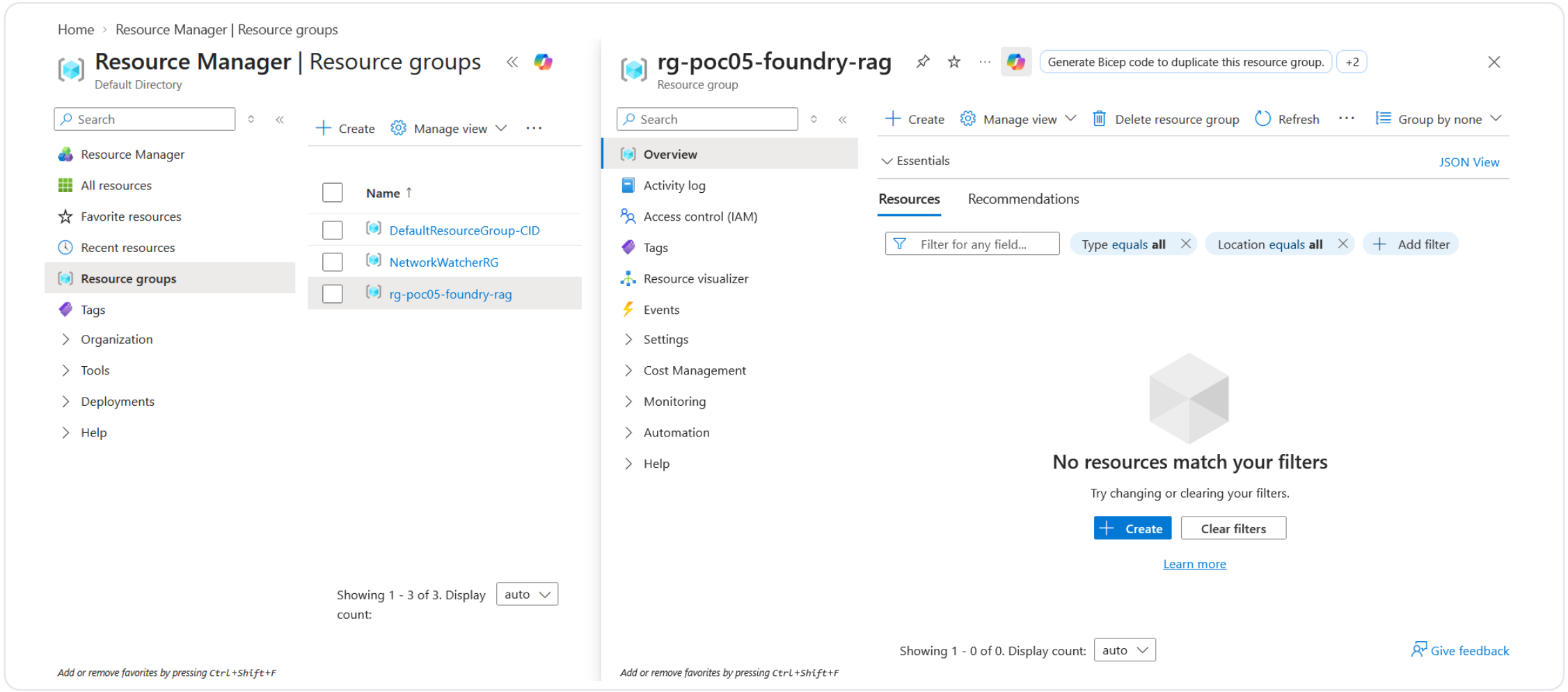
```
az group create --name rg-poc05-foundry-rag --location eastus
```

Verification command

az group show --name rg-poc05-foundry-rag --query properties.provisioningState -o tsv

Expected: Succeeded

Resource group - successful portal verification

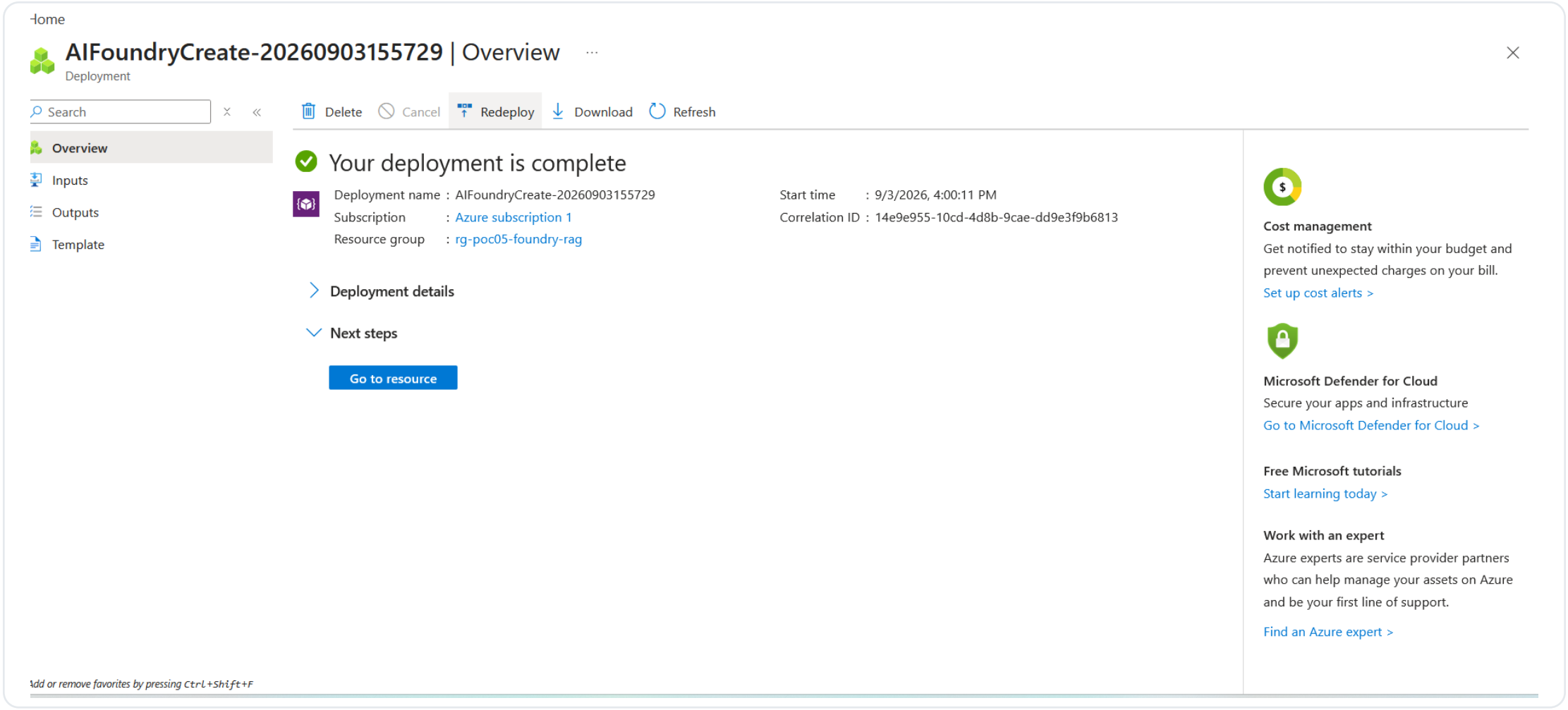


The completed resource group is visible in Azure Portal.

Step 2 - Create Microsoft Foundry resource/project

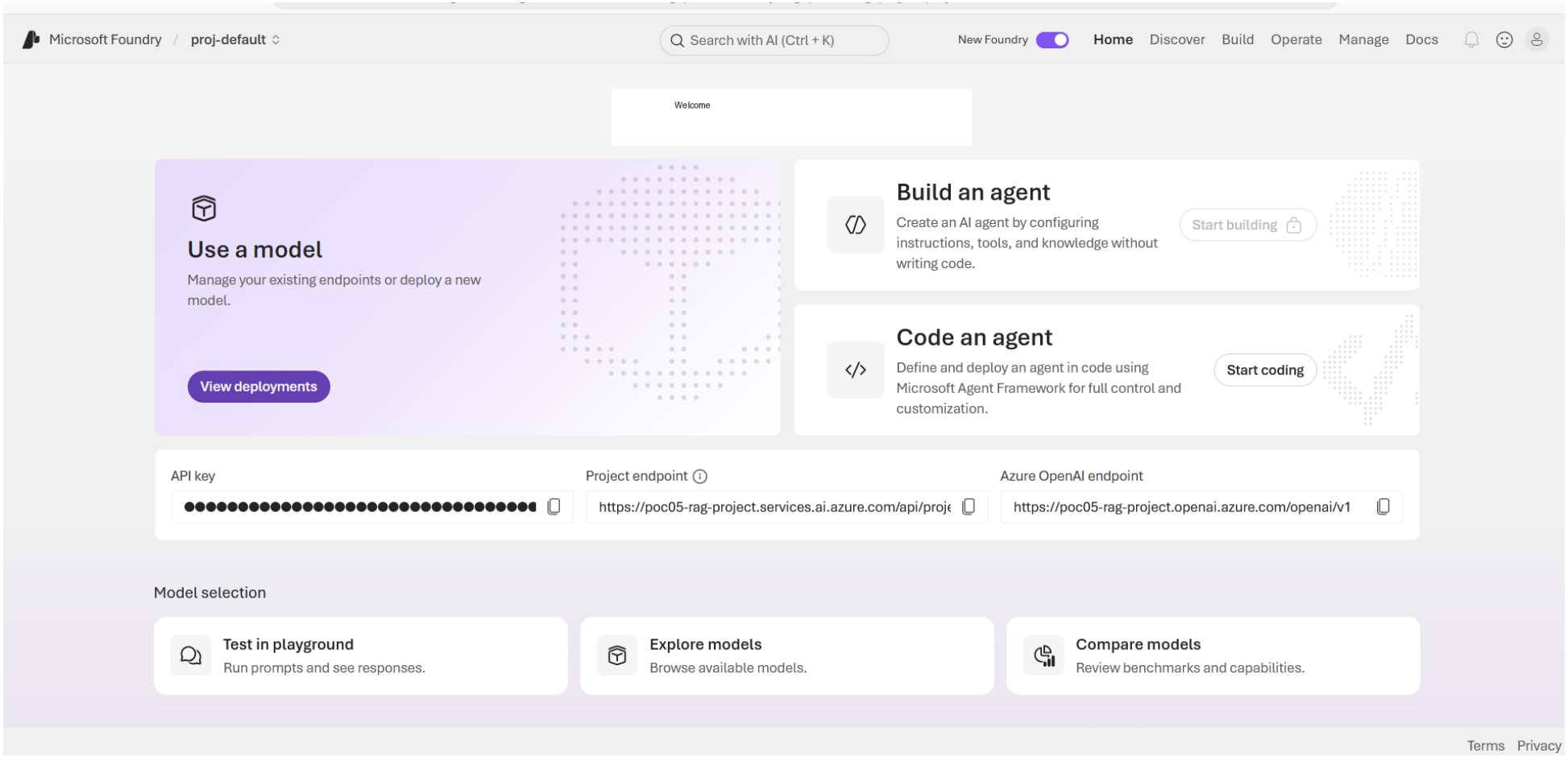
1. Open Microsoft Foundry and sign in with the Azure account connected to your subscription.
2. Use New Foundry when the portal offers the UI toggle.
3. Open the project/resource switcher and choose Create new project/resource.
4. Select resource group rg-poc05-foundry-rag.
5. Choose a Foundry resource name and a region with model capacity available to your subscription.
6. Enter the first project name, initially poc05-rag-project.
7. Keep default storage/networking/identity settings for the beginner POC unless your tenant requires different controls.
8. Create and wait for deployment to finish.

Foundry resource creation - deployment completed



Azure reports that the Foundry resource deployment completed successfully.

Foundry project - portal verification



Verify that the project home opens and the Models area is available.

Verification checkpoint

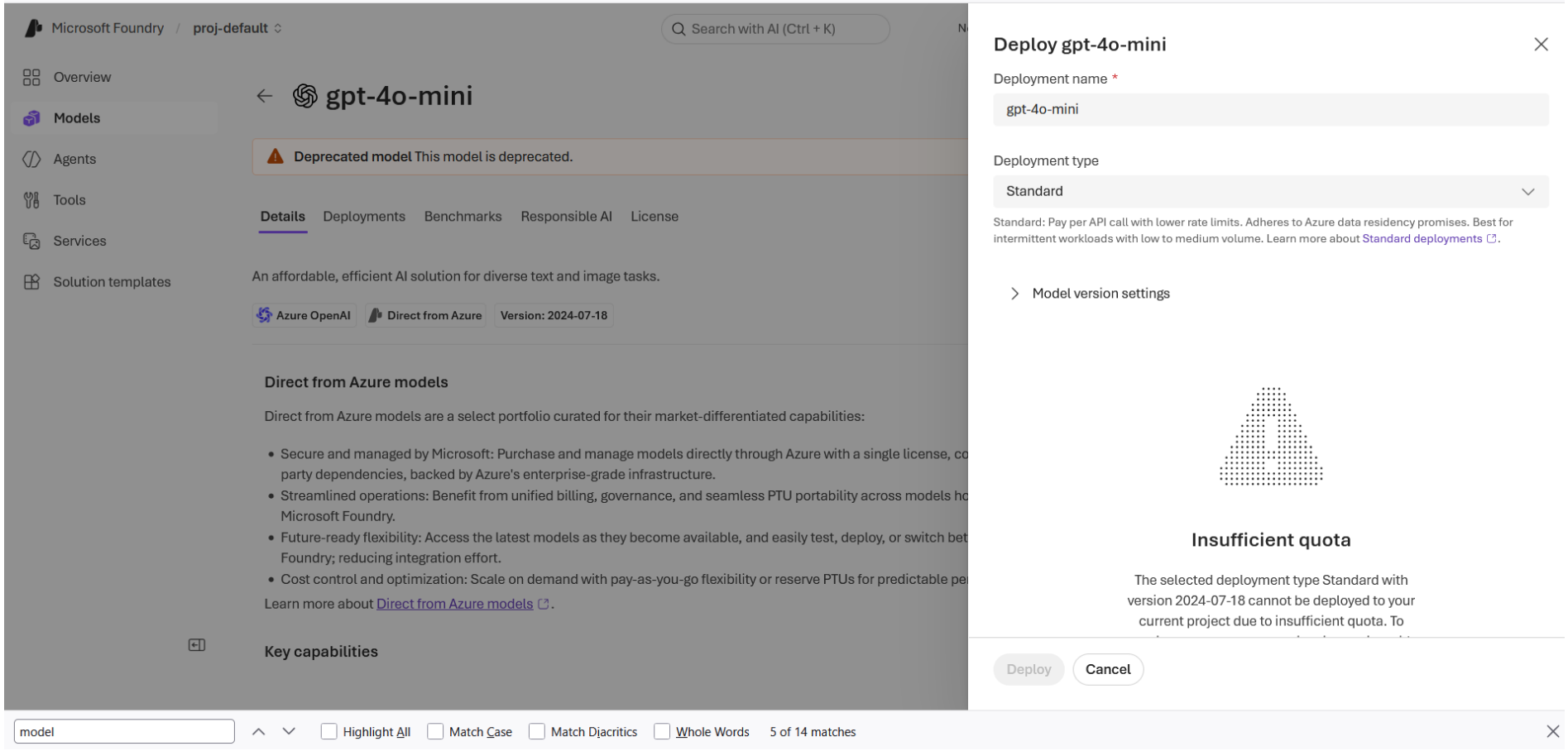
The project opens successfully, the Foundry resource is healthy, and you can browse model deployments/catalog without permission errors.

Step 3 - Deploy the chat model

1. Inside the active Foundry project, open Discover -> Models.
2. Search for a small chat-capable model available to your subscription and region.
3. Open the model card and select Deploy.
4. Use default deployment settings when quota allows; otherwise use custom settings to select an available deployment type.
5. Prefer deployment name rag-chat for a clean project, but application code must use the exact deployment name that actually exists.
6. Wait until provisioning state is Succeeded.
7. Open the playground and send: Say hello in one sentence.

> **Important**
>
> The model family name and deployment name can be different. The Python project calls the deployment name from .env.

Issue 1 - gpt-4o-mini deprecated and insufficient quota



Actual portal state: deprecated model warning and insufficient quota for the selected deployment type.

Observed problem

The Deploy button remained unavailable because the selected Standard deployment had insufficient quota. The model card also showed a deprecated version warning.

Fix 1 - Handle quota, deployment type and region

- Try another deployment type only if the portal shows quota/capacity for it, such as Global Standard or Data Zone Standard.
- If no quota is available in the current project region, create/select a project in a region offered by the portal for that model.
- The POC encountered this situation in East US and moved the active project to East US 2.
- After changing project/region, re-open the model catalog because deployment availability is tied to the active project/resource.
- Do not assume a model is usable just because its catalog card is visible; the deploy panel is the authoritative capacity check for your subscription.

> **Verification**
>
> A working path ends with a deployment whose provisioning state is Succeeded and a successful one-line playground response.

Issue 2 - Cohere deployment path did not work

What happened

A Cohere model was tried as an alternative, but the deployment path produced a model/deployment compatibility error. Third-party models can require Marketplace/serverless deployment terms and may not work through the direct Azure account path used by the current project.

Working resolution

Use a model that your subscription can deploy successfully in the active project. The completed POC used Phi-4-mini-instruct for chat and kept that exact deployment name in .env.

Do not keep retrying the same blocked path

When the portal reports DeploymentModelNotSupported or a Marketplace/serverless requirement, change the deployment path/model instead of repeatedly pressing Deploy.

Completed chat model configuration

Final chat deployment used by this POC

```
FOUNDRY_CHAT_DEPLOYMENT=Phi-4-mini-instruct
```

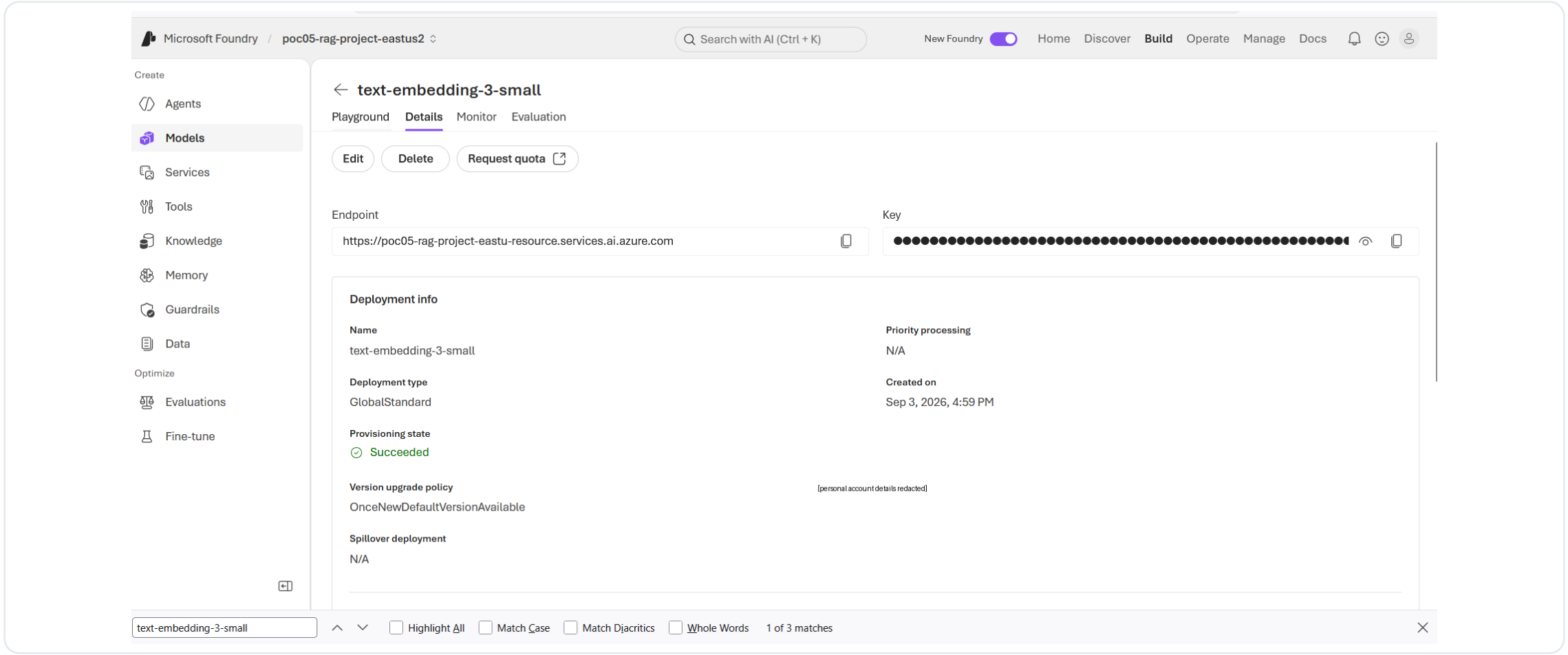
If you rename the deployment later, update .env to the new exact deployment name.

> **Playground verification**
>
> Send a tiny prompt such as "Say hello in one sentence." A coherent reply confirms the chat deployment itself works before Python integration.

Step 4 - Deploy the embedding model

1. Open Discover -> Models in the final active Foundry project.
2. Search for text-embedding-3-small.
3. Choose Deploy and select a deployment type with available quota.
4. For a clean setup, name it rag-embedding. In the completed run, the default deployment name text-embedding-3-small was kept.
5. Wait until provisioning state is Succeeded.
6. For the default text-embedding-3-small output size, configure EMBEDDING_DIMENSIONS=1536.

Embedding deployment - success and credentials



text-embedding-3-small shows GlobalStandard and provisioning state Succeeded.

Values captured for .env

Use the endpoint from the successful deployment resource, the copied key, deployment name text-embedding-3-small, and EMBEDDING_DIMENSIONS=1536.

Step 5 - Retrieve Foundry .env values

- FOUNDRY_AUTH_MODE: keep key for the first beginner run.
- FOUNDRY_OPENAI_BASE_URL: copy the final resource inference endpoint and ensure the OpenAI-compatible path ends with /openai/v1/.
- FOUNDRY_API_KEY: copy the key from the final Foundry resource/deployment details. Never commit it.
- FOUNDRY_CHAT_DEPLOYMENT: use the exact chat deployment name that succeeded.
- FOUNDRY_EMBEDDING_DEPLOYMENT: use the exact embedding deployment name that succeeded.
- EMBEDDING_DIMENSIONS: 1536 for the completed text-embedding-3-small configuration.

```
FOUNDRY_AUTH_MODE=key
FOUNDRY_OPENAI_BASE_URL=https://poc05-rag-project-eastu-resource.services.ai.azure.com/openai/v1/
FOUNDRY_API_KEY=PASTE_LOCAL_KEY_HERE
FOUNDRY_CHAT_DEPLOYMENT=Phi-4-mini-instruct
FOUNDRY_EMBEDDING_DEPLOYMENT=text-embedding-3-small
EMBEDDING_DIMENSIONS=1536
EMBEDDING_REQUEST_DIMENSIONS=false
```

Step 6 - Create Azure AI Search

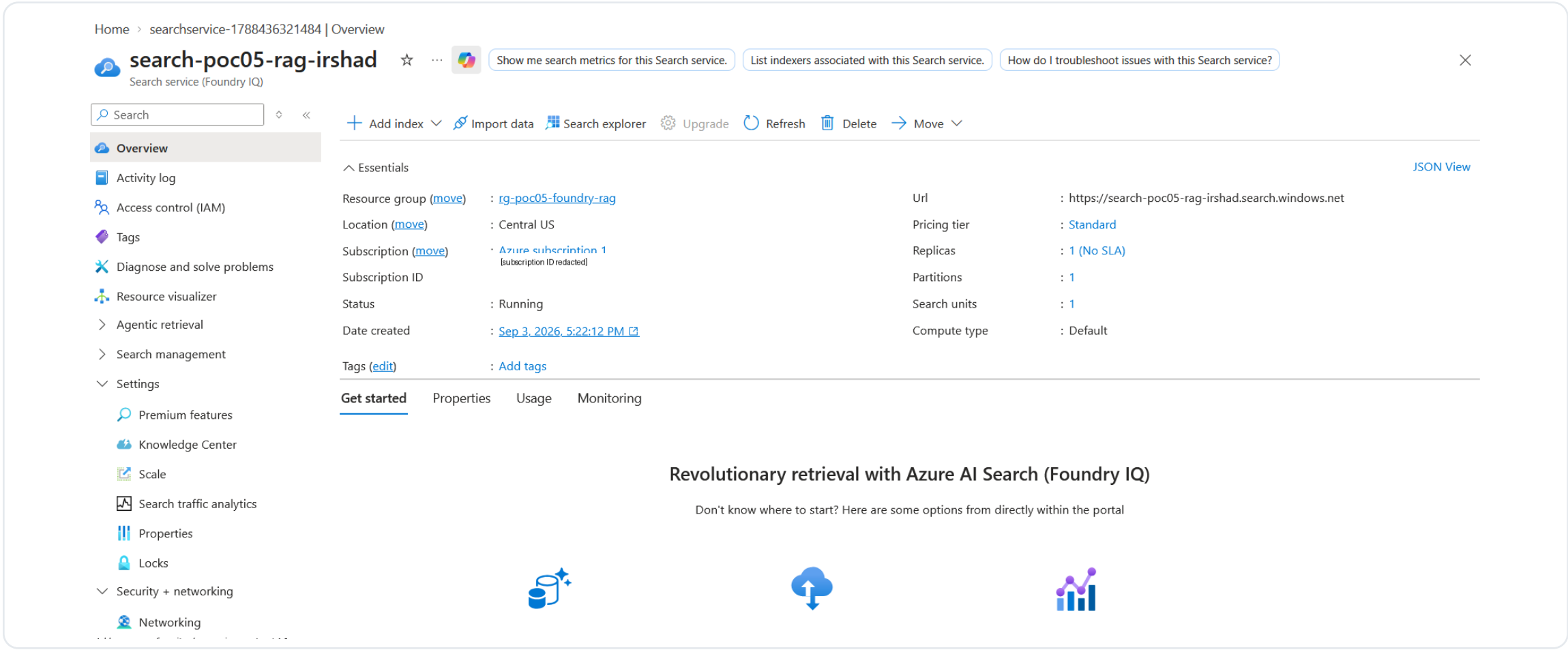
1. Open Azure Portal and search for Azure AI Search.
2. Select Create search service.
3. Use the same subscription and resource group rg-poc05-foundry-rag.
4. Choose a globally unique lowercase service name.
5. Prefer the same region as Foundry when available; a different Azure region still works because the Python app calls each service over HTTPS.
6. Choose Free when available for a small POC. If you must use a paid tier, review current costs and delete it after testing.
7. Create the service and open the Overview page.

Issue 3 - East US 2 was unavailable for Azure AI Search

- The final Foundry project used East US 2, but Azure AI Search did not offer that region during service creation.
- The POC proceeded with another available region (the successful screenshot shows Central US).
- This is acceptable for the learning POC because Foundry and Search are separate HTTPS services.
- The screenshot shows a paid Standard tier. For future rebuilds, prefer Free/Basic when available and suitable, then verify the current portal pricing/capabilities before creating.

> **Cost control**
>
> Do not leave a paid Search service or unused model deployments running after the POC if you no longer need them.

Azure AI Search - successful service creation



Service is Running. Copy the Url from Overview, then copy the Primary admin key from Settings -> Keys.

Index name is not copied from Azure

Keep AZURE_SEARCH_INDEX_NAME=poc05-rag-index. The ingestion script creates/updates that index using the schema in rag/index_schema.py.

Step 7 - Retrieve Azure AI Search .env values

- SEARCH_AUTH_MODE=key for the simplest first run.
- AZURE_SEARCH_ENDPOINT: copy the Url on the Search service Overview page.
- AZURE_SEARCH_ADMIN_KEY: open Settings -> Keys and copy the Primary admin key.
- AZURE_SEARCH_INDEX_NAME: choose/keep poc05-rag-index; it does not need to exist before ingestion.

```
SEARCH_AUTH_MODE=key
AZURE_SEARCH_ENDPOINT=https://search-poc05-rag-irshad.search.windows.net
AZURE_SEARCH_ADMIN_KEY=PASTE_PRIMARY_ADMIN_KEY_HERE
AZURE_SEARCH_INDEX_NAME=poc05-rag-index
```

> **Secret safety**
>
> .env is local configuration. Keep it in .gitignore and never paste real API keys into Markdown, screenshots, issues or commits.

Project structure - top level and RAG code

```
POC_05_FOUNDRY_RAG_AI_SEARCH_PROJECT/
|-- README.md
|-- .env.example
|-- .gitignore
|-- requirements.txt
|-- data/synthetic_docs/
|   |-- return_policy.md
|   |-- shipping_sla.md
|   |-- warranty_policy.md
|   |-- support_policy.md
|   |-- product_catalog.md
|   |-- data_dictionary_orders.md
|   |-- architecture_notes.md
|   |-- security_policy.md
|   |-- old_shipping_policy_conflict.md
|   |-- current_shipping_policy_conflict.md
|   |-- prompt_injection_test.md
|   `-- privacy_policy.md
`-- rag/
    |-- config.py
    |-- clients.py
    |-- documents.py
    |-- chunking.py
    |-- index_schema.py
    |-- ingest.py
```

Project structure - evaluation, scripts, tests and docs

```
eval/
|-- questions.json
|-- run_eval.py
|-- failure_tests.py
|-- results_template.md
`-- results/

scripts/
|-- verify_config.py
|-- smoke_test.py
`-- delete_index.py

tests/
|-- test_chunking.py
`-- test_documents.py

docs/
|-- architecture.md
|-- azure_portal_setup.md
|-- security_and_keyless_auth.md
|-- retrieval_experiments.md
|-- troubleshooting.md
|-- interview_qa.md
```

Step 8 - Prepare the local Python environment

```
cd POC_05_FOUNDRY_RAG_AI_SEARCH_PROJECT
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python -c "import azure.search.documents, fastapi, openai; print('Dependencies OK')"
```

PowerShell activation blocked?

Run: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate again. Or use Command Prompt with .venv\Scripts\activate.bat.

If ModuleNotFoundError: No module named azure

Activate the intended environment and rerun pip install -r requirements.txt.

Step 9 - Create .env (Search + Foundry)

```
# ---------- Azure AI Search ----------
SEARCH_AUTH_MODE=key
AZURE_SEARCH_ENDPOINT=https://search-poc05-rag-irshad.search.windows.net
AZURE_SEARCH_ADMIN_KEY=PASTE_PRIMARY_ADMIN_KEY_HERE
AZURE_SEARCH_INDEX_NAME=poc05-rag-index

# ---------- Microsoft Foundry ----------
FOUNDRY_AUTH_MODE=key
FOUNDRY_OPENAI_BASE_URL=https://poc05-rag-project-eastu-resource.services.ai.azure.com/openai/v1/
FOUNDRY_API_KEY=PASTE_FOUNDRY_KEY_HERE
FOUNDRY_CHAT_DEPLOYMENT=Phi-4-mini-instruct
FOUNDRY_EMBEDDING_DEPLOYMENT=text-embedding-3-small
EMBEDDING_DIMENSIONS=1536
EMBEDDING_REQUEST_DIMENSIONS=false
```

Step 9 - Create .env (RAG behavior and optional telemetry)

```
# ---------- RAG behavior ----------
CHUNK_SIZE_TOKENS=300
CHUNK_OVERLAP_TOKENS=50
TOP_K=5
MAX_OUTPUT_TOKENS=500

# ---------- Optional observability ----------
APPLICATIONINSIGHTS_CONNECTION_STRING=

# ---------- Optional logging ----------
LOG_LEVEL=INFO
```

Why EMBEDDING_REQUEST_DIMENSIONS=false?

The completed run used the default 1536-dimensional output for text-embedding-3-small. Keep false unless you intentionally request another supported dimension and rebuild the index schema to match.

Issue 4 - verify_config.py failed on Foundry base URL

```
[OK] AZURE_SEARCH_ENDPOINT configured
[OK] Azure AI Search key authentication configured
[FAIL] FOUNDRY_OPENAI_BASE_URL must be an https endpoint containing /openai/v1
[OK] Foundry key authentication configured
[OK] Embedding dimensions: 1536
[OK] Chunking configured: size=300, overlap=50

Configuration validation FAILED with 1 issue(s).
```

Cause

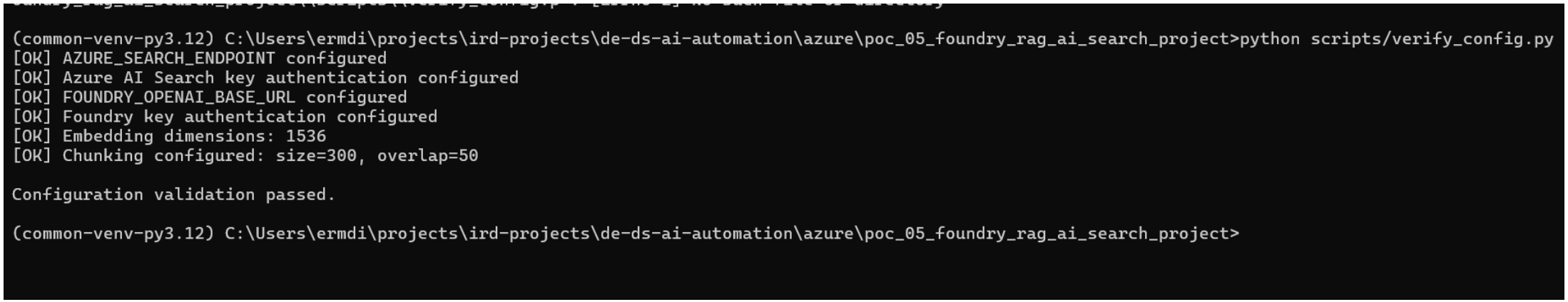
The copied Foundry endpoint only contained the resource host. verify_config.py requires an HTTPS base URL containing /openai/v1.

Fix

Set FOUNDRY_OPENAI_BASE_URL=https://poc05-rag-project-eastu-resource.services.ai.azure.com/openai/v1/ with no quotes or surrounding spaces.

Step 10 - Run configuration validation

```
python scripts/verify_config.py
```



Successful local configuration validation after adding /openai/v1/.

> **Important correction**
>
> verify_config.py validates local configuration shape/placeholders. It does not prove that the credentials have Azure permissions. Real connectivity is proven by ingestion, retrieval and RAG calls.

Step 11 - Run local unit tests

```
python -m pytest -q
```

- These tests exercise the local document parser and chunking logic without calling Azure.
- Run them before ingestion so Python/setup defects are separated from cloud configuration problems.
- Expected outcome: all tests pass.

> **Checkpoint**
> If tests fail, fix the local code/environment before investigating Azure.

Step 12 - Ingest documents into Azure AI Search

```
python -m rag.ingest
```

1. Load all synthetic Markdown files from data/synthetic_docs.
2. Read YAML front matter metadata.
3. Chunk each document using CHUNK_SIZE_TOKENS=300 and overlap=50.
4. Generate embeddings using text-embedding-3-small.
5. Create/update the Azure Search index poc05-rag-index with text and vector fields.
6. Upload all chunks and print succeeded/failed counts.

Ingestion - successful run

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project>python -m rag.ingest
Loaded 12 source documents from C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project\data\synthetic_docs
Created 12 chunks (chunk_size=300, overlap=50).
Generating embeddings...
Embeddings generated in 3.54s.
Creating/updating index 'poc05-rag-index'...
Uploading 12 chunks...
Indexed chunks: 12/12 succeeded.
Ingestion completed successfully.
Next: python -m rag.retrieve --mode vector --query "What is the return window for damaged items?"

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project>
```

Completed run: 12 source documents -> 12 chunks -> 12/12 indexed successfully.

Azure-side verification

Open the Search service -> Search management -> Indexes. Confirm poc05-rag-index exists and shows a non-zero document count.

Step 13 - Baseline vector search

```
python -m rag.retrieve --mode vector --query "What is the return window for damaged items?" --top-k 5
```

- Vector search embeds the question and finds semantically similar content_vector values.
- return_policy.md should appear near the top for this question.
- Inspect source, chunk_id, category, score and content preview.
- Record the top-k behavior before comparing hybrid retrieval.

Step 14 - Hybrid search

```
python -m rag.retrieve --mode hybrid --query "What is the return window for damaged items?" --top-k 5
```

- Hybrid retrieval sends the original text query for keyword/full-text matching and the embedding vector for semantic similarity.
- Azure AI Search fuses the ranked lists using Reciprocal Rank Fusion (RRF).
- Compare relevance with vector-only results and record observations in docs/retrieval_experiments.md.

> **Interview point**
>
> Keyword retrieval is strong for exact identifiers and terms; vectors help when wording differs. Hybrid combines both strengths.

Step 15 - Metadata filtering

```
python -m rag.retrieve --mode hybrid --category policy --query "How quickly should standard orders ship?"
```

- The category filter limits which indexed documents are eligible.
- Filters can improve relevance and become important for enterprise security trimming or tenant/document access rules.
- For this test, only documents with category=policy should be returned.

Step 16 - Run the full grounded RAG answer

```
python -m rag.ask --query "What is the return window for damaged items?"
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project>python -m rag.ask --query "What is
the return policy window?"

ANSWER
------
The return policy window for unopened standard products is within 30 calendar days of delivery. This is specified in the "Returns and Damaged Items Policy"
from the context provided. Damaged or defective items must be reported within 14 calendar days of delivery, and after the damage report is accepted, the cus
tomer has 21 calendar days to send the damaged item back using the provided return label.

RETRIEVED SOURCES
-----------------
- return_policy.md#return_policy-0001
- current_shipping_policy_conflict.md#current_shipping_policy_conflict-0001
- old_shipping_policy_conflict.md#old_shipping_policy_conflict-0001
- warranty_policy.md#warranty_policy-0001
- privacy_policy.md#privacy_policy-0001

Latency: 6418.21 ms

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project>
```

Direct CLI answer includes grounded text, source identifiers and latency.

> **Expected behavior**
>
> The answer must come from retrieved context and cite the source/chunk. Retrieved document text is treated as untrusted data and cannot override system instructions.

Step 17 - Test an unsupported question

```
python -m rag.ask --query "Who won the 2032 World Cup?"
```

> **Expected fallback**
> I don't have enough information.

- The corpus does not contain the answer, so the system should not confidently invent one.
- Low-score retrieved chunks may still exist, but generation should remain grounded to supported evidence.
- This is one of the most important anti-hallucination checks in the POC.

Step 18 - Run retrieval and RAG evaluation

```
python -m eval.run_eval
```

- The project includes 10 evaluation questions with expected source documents.
- Results are written as timestamped JSON under eval/results/.
- Metrics include retrieval_hit_at_k, citation_correct, answer_grounded_heuristic, unsupported_answer and latency_ms.
- Use the metrics to compare vector vs hybrid settings, chunking and top-k changes.

Step 19 - Run failure-case tests

```
python -m eval.failure_tests
```

1. Question not present in the corpus.
2. Ambiguous question.
3. Conflicting documents with different effective content.
4. Prompt-injection text embedded inside a retrieved document.

> **What to record**
>
> Record which document was retrieved, whether the citation was correct, whether the answer stayed grounded, and how the system behaved when evidence conflicted or attempted to manipulate the prompt.

Step 20 - Run the end-to-end smoke test

```
python scripts/smoke_test.py
```

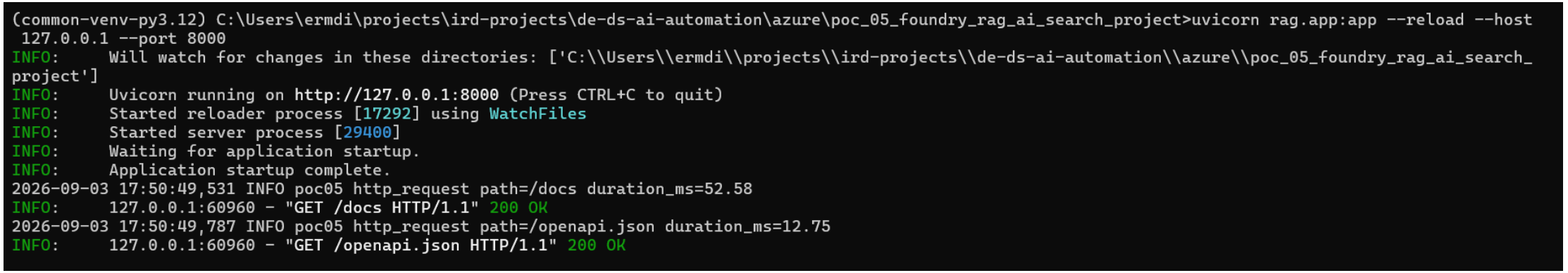
```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project>python scripts/smoke_test.py
1/3 Vector retrieval
   top source: return_policy.md
2/3 Hybrid retrieval
   top source: return_policy.md
3/3 Grounded generation
   answer: The return window for damaged items, according to the provided context, is as follows:

- Damaged or defective items must be reported within 14 calendar days of delivery.
- After the damage report is accepted, the customer has 21 calendar days to send the damaged item back using the provided return label.

This information is found in the "Returns and Damaged Items Policy" from the source "return_policy.md" with the chunk_id "return_policy-0001".

SMOKE TEST PASSED

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_05_foundry_rag_ai_search_project>
```

The smoke test proves vector retrieval, hybrid retrieval and grounded generation in one short run.

> **Success indicator**
>
> SMOKE TEST PASSED

Step 21 - Start the FastAPI application

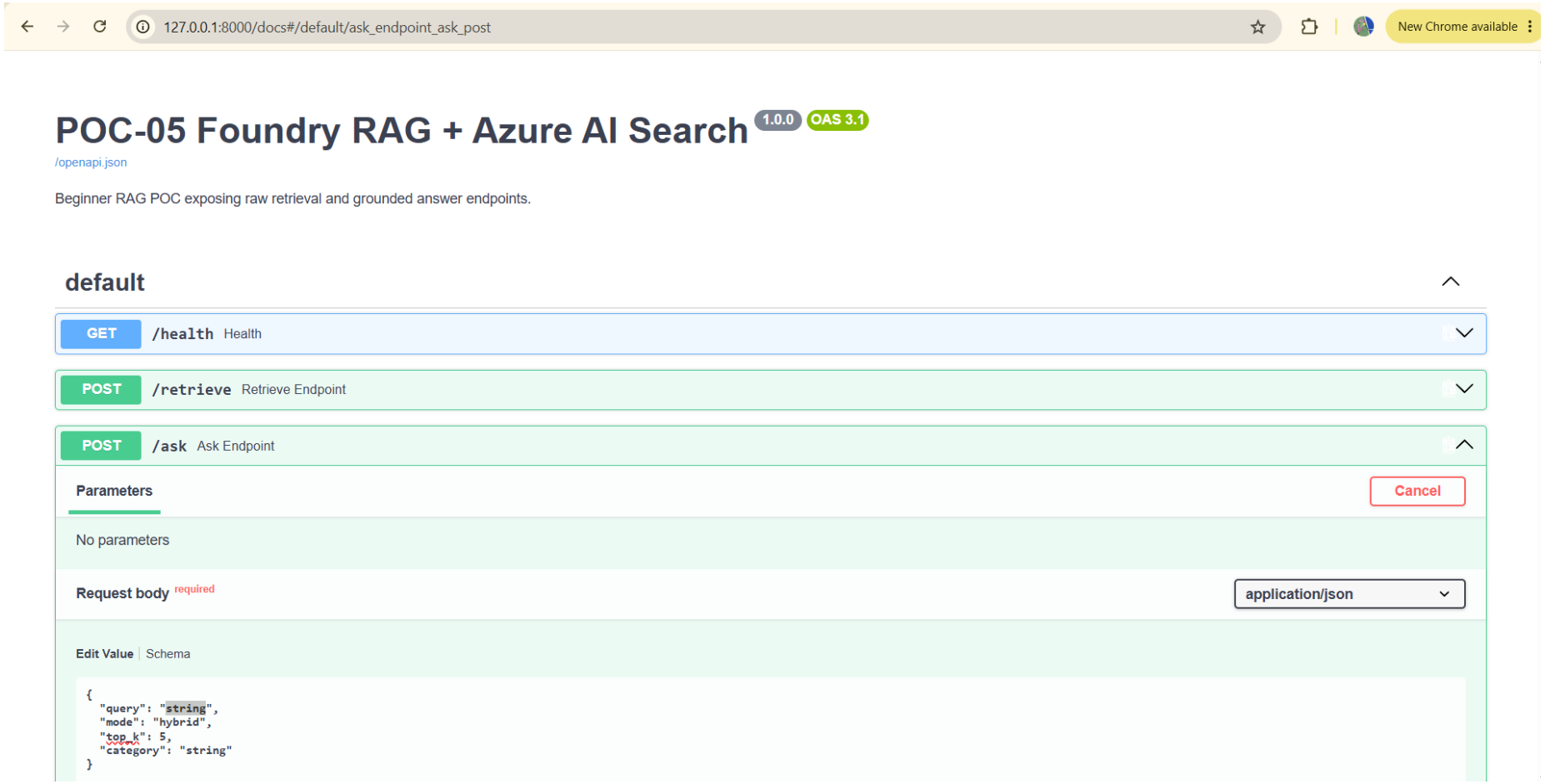
```
uvicorn rag.app:app --reload --host 127.0.0.1 --port 8000
```



Uvicorn is running on the local development address.

> **Correct framework**
>
> rag/app.py is a FastAPI application. Use uvicorn for this project; do not use a Streamlit launch command.

Step 22 - Verify FastAPI routes in Swagger

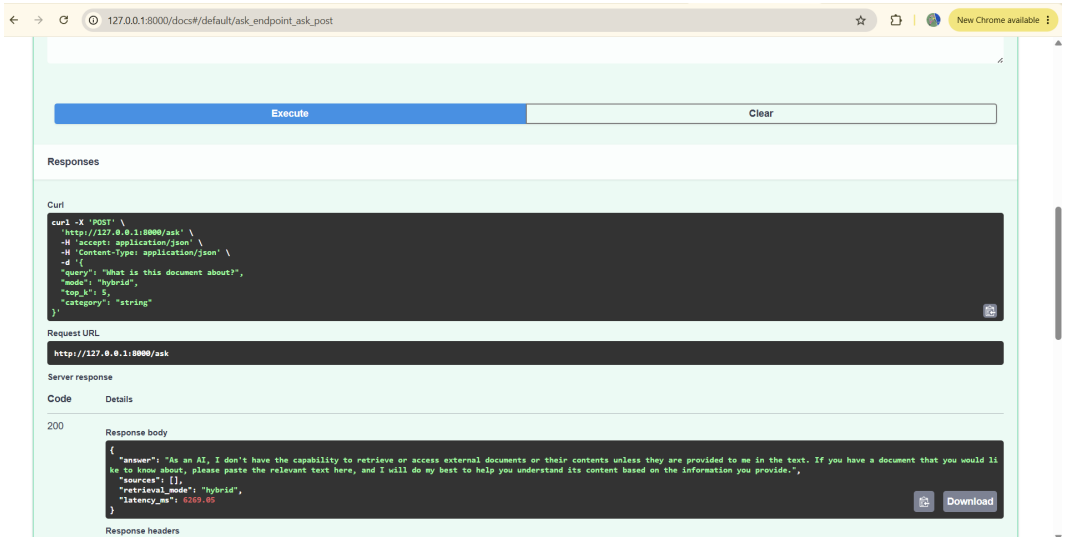


Open the local /docs page to verify GET /health, POST /retrieve, and POST /ask are available.

```
http://127.0.0.1:8000/docs
```

Step 23 - Test POST /ask

```
{
  "query": "What is the return window for damaged items?",
  "mode": "hybrid",
  "top_k": 5,
  "category": null
}
```



Swagger Execute returns a successful grounded response with answer, sources, retrieval mode and latency.

Step 24 - Verify /retrieve and /health

- POST /retrieve returns raw retrieval chunks so you can inspect search quality separately from LLM generation.
- GET /health returns status plus the configured search index and authentication modes.
- Use /retrieve when debugging relevance; use /ask when validating end-to-end grounded generation.

```
GET  http://127.0.0.1:8000/health
POST http://127.0.0.1:8000/retrieve
POST http://127.0.0.1:8000/ask
```

Optional - Application Insights observability

1. Create an Application Insights resource if you want cloud telemetry.
2. Copy its connection string into APPLICATIONINSIGHTS_CONNECTION_STRING in .env.
3. Restart the FastAPI process.
4. The application enables Azure Monitor OpenTelemetry when the connection string is present.
5. Request duration and failures are still logged to the console when the variable is blank.

```
APPLICATIONINSIGHTS_CONNECTION_STRING=InstrumentationKey=...;IngestionEndpoint=...
```

Optional - Switch from keys to Microsoft Entra ID

- First prove the key-based POC works.
- For Azure AI Search, assign the required Search Service Contributor / Search Index Data Contributor / Search Index Data Reader roles as appropriate.
- For Foundry model access, assign the role required by your deployment path.
- Run az login locally.
- Change SEARCH_AUTH_MODE=entra and FOUNDRY_AUTH_MODE=entra, then remove the corresponding keys from .env.
- The code uses DefaultAzureCredential, which can also resolve managed identity when the app is hosted in Azure.

GitHub safety before push

```
git status
git check-ignore .env
git grep -n "FOUNDRY_API_KEY"
git grep -n "AZURE_SEARCH_ADMIN_KEY"
```

- git check-ignore .env should print .env, proving the secret file is ignored.
- Never commit API keys or copied portal secrets.
- Use only synthetic documents in the public POC repository.
- If a key appears in a screenshot, issue, Markdown file or commit, rotate it immediately.

Cleanup and cost control

```
# Delete only the Search index
python scripts/delete_index.py

# Delete the entire isolated resource group when finished
az group delete --name rg-poc05-foundry-rag --yes --no-wait
```

> **Caution**
> Delete the resource group only when you are sure it contains nothing important outside this POC.

- Delete unused paid model deployments/resources after testing.
- Review paid Azure AI Search tiers before leaving them provisioned.
- Keep the resource group isolated so cleanup is predictable.

Complete command runbook - setup and ingestion

```
cd POC_05_FOUNDRY_RAG_AI_SEARCH_PROJECT
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env
# Edit .env and save it
python scripts/verify_config.py
python -m pytest -q
python -m rag.ingest
```

Complete command runbook - retrieval and RAG

```
python -m rag.retrieve --mode vector --query "What is the return window for damaged items?"
python -m rag.retrieve --mode hybrid --query "What is the return window for damaged items?"
python -m rag.retrieve --mode hybrid --category policy --query "How quickly should standard orders
ship?"
python -m rag.ask --query "What is the return window for damaged items?"
python -m rag.ask --query "Who won the 2032 World Cup?"
```

Complete command runbook - evaluation and API

```
python -m eval.run_eval
python -m eval.failure_tests
python scripts/smoke_test.py
uvicorn rag.app:app --reload --host 127.0.0.1 --port 8000
# Open http://127.0.0.1:8000/docs
```

Troubleshooting - Azure and model deployment

az is not recognized

Install Azure CLI separately, reopen the terminal, and restart Windows if PATH has not refreshed.

Deprecated model warning

Prefer a newer supported model/version when available. A deprecated card is a warning; deployment still depends on quota/capacity.

Insufficient model quota

Select a deployment type/region with actual quota, choose another model, or request quota. Re-check the active project before deploying.

Cohere / DeploymentModelNotSupported

Third-party models can require a different Marketplace/serverless path. Use a compatible model/deployment path; the completed run used Phi-4-mini-instruct.

Azure AI Search desired region unavailable

Choose another available region. Cross-region HTTPS calls still work for the POC.

Troubleshooting - configuration and Search

Foundry base URL validation fails

Ensure FOUNDRY_OPENAI_BASE_URL begins with https:// and contains /openai/v1/. Do not use the project API endpoint.

Azure AI Search 401

Check the endpoint, the admin key, whether the key came from the same Search service, and whether key authentication is enabled.

Azure AI Search 403 with Entra ID

Check RBAC role assignments and allow time for role propagation.

Foundry 401/403

Check the base URL, key/az login, model-access RBAC role and networking/firewall configuration.

Model/deployment not found

Use the deployment name that exists in the portal, not merely the model family name.

Troubleshooting - vector index and retrieval

Embedding dimension mismatch

Set EMBEDDING_DIMENSIONS to the actual embedding length. If the index schema already exists with another vector size, delete the index and ingest again.

Search index schema cannot be updated

Run python scripts/delete_index.py, then python -m rag.ingest.

Free Search tier unavailable

Check whether an old free service already exists and try another region. If using a paid tier, review cost and delete it after the POC.

Retrieval quality is poor

Compare hybrid vs vector, top-k 3/5/10, chunk size, overlap, metadata filters, titles and whether the same embedding model/config is used for indexing and querying.

RAG uses outside knowledge

Use stronger retrieval confidence checks, grounded prompts, citation validation, evaluation and failure testing in production.

POC completion checklist - Azure and data path

- Resource group created and provisioning state is Succeeded.
- Foundry resource/project created.
- Chat model deployment succeeds and playground responds.
- Embedding deployment succeeds.
- Azure AI Search service created.
- .env configured locally and ignored by Git.
- verify_config.py passes.
- Local unit tests pass.
- rag.ingest completes with no failed chunks.
- poc05-rag-index exists and has non-zero documents.

POC completion checklist - retrieval, RAG and API

- Vector query returns a relevant source.
- Hybrid query works and is compared with vector-only retrieval.
- Metadata filter works.
- RAG answer contains source identifiers.
- Unsupported question falls back instead of inventing facts.
- 10-question evaluation runs and writes results.
- Failure tests run.
- smoke_test.py passes.
- FastAPI /health, /retrieve and /ask work from Swagger.
- Optional: Application Insights telemetry and Entra ID/keyless authentication are tested.
- Unused paid resources are deleted when finished.

Interview revision - retrieval fundamentals

Q1. Vector search vs keyword search?

Keyword search matches lexical text and is strong for exact terms, IDs, names and phrases. Vector search compares embeddings and can retrieve semantically similar text even when wording is different.

Q2. Why hybrid retrieval?

Keyword and vector retrieval have complementary strengths. Hybrid runs both and fuses ranked results, helping exact identifiers and natural-language intent in the same query.

Q3. What is RRF conceptually?

Reciprocal Rank Fusion combines ranked result lists. Documents placed high in one or more lists receive more weight. Azure AI Search uses RRF to merge full-text and vector results for hybrid queries.

Interview revision - chunking and metadata

Q4. Chunk-size trade-offs?

Small chunks are precise but may lose context and increase vector count. Large chunks preserve context but may contain unrelated material and waste tokens. Overlap protects boundary context but increases embedding/storage work.

Q5. Why metadata filters?

Filters reduce the eligible search space using structured fields such as category, tenant or document type. They improve relevance and are essential for enterprise authorization/security trimming.

Q6. Why does embedding dimension matter?

Every vector in an Azure AI Search vector field must match the dimensions configured in the index schema, and query vectors must use the same embedding space/configuration.

Interview revision - quality and hallucination control

Q7. How do you measure RAG quality?

Separate retrieval and generation. Retrieval can use hit@k, recall@k, precision, MRR and nDCG. Generation should check groundedness, correctness, citation correctness, unsupported-answer rate, latency, cost and task success.

Q8. How do you reduce hallucinations?

Use strong retrieval, explicit grounded prompting, fallback behavior for insufficient evidence, citations, trusted context, evaluation gates, document version control and continuous failure testing.

Q9. How would you secure enterprise RAG?

Use Entra ID/managed identity, least-privilege RBAC, private endpoints where needed, Key Vault for remaining secrets, authorization filters/security trimming, safe logging, monitoring, audit controls and prompt-injection defenses.

What this POC proves

- End-to-end synthetic document ingestion and metadata-aware chunking.
- Embedding generation and vector indexing in Azure AI Search.
- Vector-only and hybrid retrieval with relevance comparison.
- Grounded answer generation with source identifiers.
- Unsupported, ambiguous, conflicting and prompt-injection failure testing.
- Retrieval/RAG evaluation metrics and latency measurement.
- FastAPI serving with raw retrieval and grounded-answer endpoints.
- Practical Azure troubleshooting for quota, model availability, region differences and endpoint configuration.
- Secure configuration patterns with local .env, optional Entra ID/managed identity and optional Application Insights.

Final reference point

The successful screenshots in this guide prove the main POC path: resources created, configuration validated, 12/12 chunks indexed, smoke test passed, API started, Swagger loaded and /ask returned a grounded response.